Midterm Technique Report

Part 1 — Small web service

I used the Node.js starter code provided by the instructor and kept its endpoints and counter logic. I wrote a Dockerfile to build the image cs554-midterm:v1.

GET / returns the service name, message, hostname, and this process's request count. GET /health returns HTTP 200 and does not increase the count.

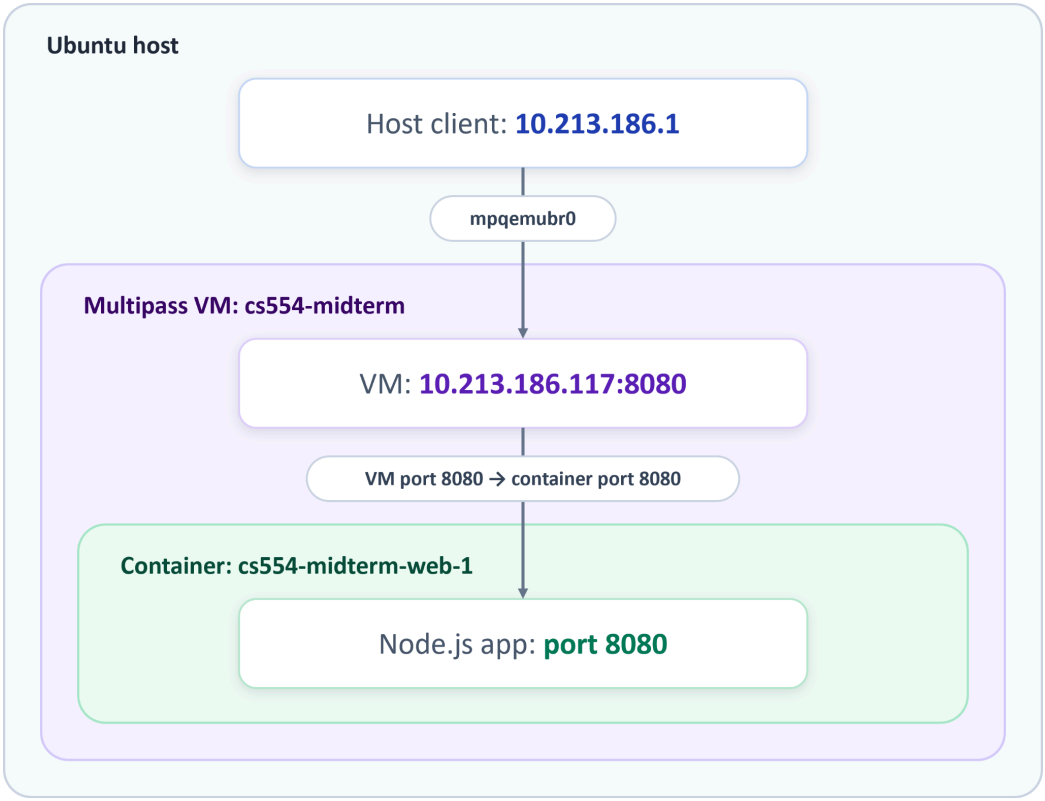
I used `PORT`, `SERVICE_NAME`, and `MESSAGE` to set the port, service name, and message for the deployment. The app writes startup and request logs to standard output.

Part 2 — Docker Compose inside a Linux VM

I ran Docker Engine and Compose in the Multipass VM cs554-midterm (Ubuntu 24.04). Compose used the same image and set the environment variables. The application inside the container can be accessed through port 8080 on the VM.

Part 2 — Network and request path

The host reached VM 10.213.186.117 through the private bridge mpqemubr0, the host IP is 10.213.186.1. No extra host-to-VM port forwarding was used. The application runs inside the container, which uses the VM's Linux kernel. The path diagram is shown below:



Part 2 — Results and interpretation

- Inside the VM, `/` and `/health` both returned HTTP 200.
- From the host, `http://10.213.186.117:8080/` returned HTTP 200 with the same hostname.
- Compose showed the container `cs554-midterm-web-1` as `Up`. Logs showed startup and successful requests.

Before running `docker compose restart web`, the last response showed a request count of `3`. After the restart, the hostname stayed the same, and the application's in-memory counter reset to `0`. The first successful `GET /` request after the restart increased the count to `1`.

Part 3 — Kubernetes deployment

I ran a single-node k3d cluster in Docker on the Ubuntu host. I imported the same `cs554-midterm:v1` image into the node. Deployment `cs554-web` used two replicas, and a ClusterIP Service selected Pods labeled `app: cs554-web` on port 8080. I used `sudo kubectl port-forward service/cs554-web 8081:8080` and sent requests from the host to `localhost:8081`. Both `/` and `/health` returned HTTP 200. This port-forwarding session provided host access to one Pod selected through the Service. It did not demonstrate round-robin traffic distribution or normal ClusterIP routing.

After I deleted Pod `qpb6g`, Kubernetes created `7j9cb`, while `jnnff` remained. Both Pods were Ready, and the Deployment returned to `2/2`. Its ReplicaSet created the replacement to maintain the desired two replicas.

VM/Compose versus Kubernetes comparison

Compose ran one application container inside the VM. When I restarted the service, the existing container's application process restarted. The hostname stayed the same. The last response before the restart showed a request count of 3. Restarting the application reset its in-memory counter to 0, and the first successful GET / afterward increased it to 1.

Kubernetes ran two application Pods through a Deployment. After I deleted one Pod, its ReplicaSet automatically created a replacement to maintain the desired two replicas. The other Pod kept running, and the Deployment returned to '2/2' Ready. The Compose experiment showed a manual restart of an existing instance, while Kubernetes demonstrated automatic replacement of a deleted instance.

CS 554 Additional Part— Resources and readiness experiment

Each application container requests 100m CPU and 64Mi memory, with limits of 500m CPU and 128Mi memory. These are modest starting values for this small HTTP service, with room above the requests for bursts. Requests guide scheduling; limits bound resource use. CPU can be throttled, while exceeding the memory limit can cause an out-of-memory termination.

The normal readiness probe checks `/health` on port 8080. Failed readiness makes a Pod ineligible for normal Service traffic without restarting its container. This differs from the Compose restart, which restarted the application process and reset its counter.

For the experiment, I temporarily used an exec probe. It normally checked `/health`, but a `/tmp/not-ready` file inside one Pod switched its check to the non-serving `/not-ready` path. I first recorded both Pods Ready and both EndpointSlice entries `ready: true`. Creating the file only in `hbg68` changed that Pod to `0/1 Running`, with zero restarts. Its endpoint became `ready: false`; the other stayed ready.

The NotReady Pod still returned HTTP 200 from its local `/health`. An in-cluster request to the Service returned HTTP 200 from the Ready Pod `g89vp`, consistent with the endpoint conditions. Removing the file(/tmp/not-ready) restored the same Pod to Ready and its endpoint to `ready: true`. During the marker-file failure and recovery steps, Pod names, IPs, UIDs, and restart counts stayed unchanged. Restoring the original Deployment template afterward triggered a rollout and replaced the Pods.

Part 4 — Controlled troubleshooting

Before the failure, Deployment `cs554-web` had two Ready replicas, the Service had two backend addresses, and an HTTP request returned 200 and JSON. The Node.js client ran inside an existing Pod and accessed `http://cs554-web:8080/`, so the request depended on the Service's selector.

I changed the live Service selector from `app: cs554-web` to `app: cs554-web-wrong`. The same client request failed with `TypeError: fetch failed` and exit code 1. Both Pods remained `1/1 Running`, their labels stayed `app=cs554-web`, and their restart counts stayed at 1. The Service's `spec.selector` no longer matched those labels, and the EndpointSlice showed `<unset>` for `ENDPOINTS`. These observations identified the selector mismatch as the cause.

I restored the selector to `app: cs554-web`. Both backend addresses, `10.42.0.19` and `10.42.0.20`, returned, and the same request again received HTTP 200 and JSON. Healthy Pods alone do not prove that a client can reach an application through a Service: the Service must also select the correct Pods and have eligible backend endpoints.

Comment: You can find supporting evidence for all the operations and results I mentioned above in evidence/EVIDENCE_PARTS.md.

Part 5 — OpenStack deployment analysis

Resources and choices - I will deploy `cs554-midterm:v1` with Compose on an Ubuntu 24.04 LTS amd64 VM in the cloud which is provided by the professor. I'll choose a flavor near 1 vCPU, 2 GB of memory, and a 20 GB root disk. I think this is a moderate initial configuration for Ubuntu, Docker, and this service.

The administrator will provide project access, an approved Glance image, available flavors, and an external network with a floating IP pool. Unless provided, I will create a project network and DHCP-enabled private IPv4 subnet, then a router with an external gateway and subnet interface. And I will prepare the security group and import my SSH public key as a key pair

before launching the VM on that network. Finally, I will allocate a floating IPv4 address and associate it with the VM's Neutron port. [1, 3]

A separate Cinder data volume is unnecessary: the root disk holds the OS and deployment files. The counter lives in process memory; attaching a volume will not preserve it without application changes.

Service responsibilities - Keystone authenticates the user and provides project-scoped tokens and service endpoints. Glance supplies the Ubuntu VM image. Nova uses Placement to select a compute host and directs the hypervisor to start the VM. Neutron manages its port, fixed IP, networking, and security groups. [1]

To install Docker Engine and Compose online, the VM needs working DNS and access to the package repositories. It will normally access these repositories through the router using SNAT. If the VM cannot access them, I will transfer the installation packages for Ubuntu 24.04 (amd64), along with all required dependencies, to the VM and install them offline. [3, 4, 5]

I will transfer and load the saved application image, confirm that its architecture is amd64, and copy `compose.yaml`. Its `image: cs554-midterm:v1` and `pull_policy: never` use the loaded image, including its Node base layers, without downloading from Docker Hub. Compose starts `web` with the existing environment variables and `8080:8080` mapping. [5]

Request path - A client that can reach the cloud's external network will send a request to `http://FLOATING_IP:8080/`. Neutron uses routing and destination NAT to direct the request to the VM's private IP. If the security group and the VM/Docker firewall rules allow the traffic, it reaches port 8080 on the VM. Docker then forwards it to port 8080 inside the container, where the Node.js application is running. The response returns along the same path. [4]

The security group will allow inbound IPv4 TCP 8080 from the professor's approved source network. SSH needs a separate TCP 22 rule from my administration address. Optional ICMP supports ping; SSH or ping success does not prove HTTP access. Keystone, Glance, and Nova do not carry application traffic. [1, 3, 4]

Failure analysis - If the VM is running but external HTTP fails, I will first inspect container status and logs, then use the container's existing Node.js `fetch` to request `/` and `/health`. I will save HTTP statuses, response bodies, and logs.

Next, I will test the VM's loopback and private addresses, recording published ports, listening addresses, firewall rules, and HTTP results. Finally, I will inspect Neutron port status, security-group rules, floating IP association, router interfaces, and external gateway, saving these settings and the external client's error. [1]

Operational complexity - Operating OpenStack requires coordinating hardware, hypervisors, databases, messaging, APIs, networking, storage, and compatible dependencies. Using a working cloud mainly involves resource choices and application configuration.

The course material (The OpenStack Lab That Turned Into a Systems Debugging Exercise) describes Placement appearing active while its API returned HTTP 500 because of an `oslo.policy` mismatch. Nova conductors failed and `nova-compute` could not register. Compatible dependencies and service restarts restored the chain. This was an infrastructure problem, distinct from an application port or selector error. [2]

Comparison - One VM with Compose is enough for this small service if I only need one instance and can handle problems manually. In my local experiment, Kubernetes maintained two replicas, replaced a deleted Pod, and excluded a Running but NotReady Pod from ready Service backends without restarting its application.

References

[1] CS554 course slides. OpenStack — Architecture.
[2] CS554 course material. The OpenStack Lab That Turned Into a Systems Debugging Exercise.
[3] OpenStack. OpenStack Installation Guide: Launch an instance,
https://docs.openstack.org/install-guide/launch-instance.html.
[4] OpenStack. OpenStack Networking,
https://docs.openstack.org/neutron/latest/admin/intro-os-networking.html.
DNS Resolution for Instances,
https://docs.openstack.org/neutron/latest/admin/config-dns-res.html.
[5] Docker Documentation. Install Docker Engine on Ubuntu,
https://docs.docker.com/engine/install/ubuntu/,
docker image save, https://docs.docker.com/reference/cli/docker/image/save/.